Powershell

About powershell

- What is powershell
  - WMF and .NET
  - Host applications
    - Console
    - ISE and WPF
    - Powerpick
    - Web
- Powershell versions
  - Determine version
  - Different operating systems

Executing commands

- x86 vs x64 versions
- .\script.ps1
- Opening scripts in new tab in ISE
- Tab complete and command history
- Native commands from command interpreter
  - Unsupported commands in ISE
- Command precedence
  - Alias
  - Function
  - Cmdlet
  - Native windows command

Special characters and operators

- What are special characters ?
- What would the following characters do ?
  - Open parentheses (()
  - Open curly brace ({)
  - Open square bracket ([)
  - Open angle bracket or greater than symbol (<)
  - Vertical (broken) bar (|)
  - Double quote (")
  - Single quotation mark (')
  - Grave accent / backtick (`)
  - At sign (@)
  - Hash (#)
  - Dollar sign ($)
  - Slash (/)
  - Backslash (\ )
  - Underscore (_)
  - Star (*)
  - Question mark (?)
  - Exclamation mark (!)
  - Percent sign (%)
  - Ampersand (&)
  - Caret (^)
  - Tilde (~)
  - Colon (:)
  - Semicolon (;)

Open / Close parentheses

- Plain parentheses act as a *grouping expression* to encapsulate a single cmdlet or a single sequence, where a sequence might be something piped to something else that is piped to something else again. The output of a sequence comes only from the terminal command or cmdlet of the pipeline, i.e. a single source.

```
PS C:\> (get-service -name win*).name
WinDefend
WinHttpAutoProxySvc
Winmgmt
WinRM
PS C:\> _
```

- An important aspect of a grouping expression is the encapsulation. The content of the expression is executed until completion and the result is then returned to be operated on by something else. Taking advantage of this encapsulation is what allows you to update a file in-place (i.e. without a temporary file or file renaming)

```
PS C:\> (Get-Content somefile.txt) | %{ $_ -replace 'aa', 'bb'} | Set-Content somefile.txt
```

- When a PowerShell construction requires multiple sets of brackets, the parenthesis usually comes first. Parenthesis brackets also come first in importance because these (curved) brackets are used for what we can call compulsory arguments or control structures

```
PS C:\> ForEach ($item in $allitems)
>> {
>> Write-Host $item.name
>> Write-Host $item.description
>> }
```

Open / Close curly brace

- The most common use of braces is to define a scriptblock.

```
PS C:\> get-service -name win* | where {$_.status -eq 'running'}

Status   Name               DisplayName
------   ----               -----------
Running  WinDefend          Windows Defender Service
Running  WinHttpAutoProx... WinHTTP Web Proxy Auto-Discovery Se...
Running  Winmgmt            Windows Management Instrumentation


PS C:\>
```

```
PS C:\> if (Condition)
>> {
>> Do some commands
>> }
>> else
>> {
>> do other commands
>> }
>>
```

- In addition You can use curly braces to define variable names with arbitrary characters in the name (for example spaces)
  - Sometimes generators always put variables inside the curly braces to ensure that the variable name is valid

```
PS C:\> ${variable}="Hello World1"
PS C:\> $variable
Hello World1
PS C:\> ${variable and some additional text}="Hello World2"
PS C:\> ${variable and some additional text}
Hello World2
PS C:\>
```

Open / Close square bracket

- The standard use case for square brackets is arrays to access n-th object

```
PS C:\> $variable=1..10
PS C:\> $variable[3]
4
PS C:\> $variable[-2]
9
```

- In powershell square brackets can also used as regex or “Like”

```
PS C:\> get-service [e]*

Status   Name               DisplayName
------   ----               -----------
Stopped  Eaphost            Extensible Authentication Protocol
Stopped  EFS                Encrypting File System (EFS)
Running  EventLog           Windows Event Log
Running  EventSystem        COM+ Event System
```

- You can also use square brackets to return .NET classes

```
PS C:\> $x=3.14
PS C:\> $x -as [int]
3
PS C:\>
```

- In regular expressions square brackets match a set or specific characters
  - [a-l]ook matches “book”, “cook” but does not match “took”
  - [bc]ook matches “book”, “cook” but not “look”

Greater than

- Powershell uses greater than “ > “ as output redirector but compared to regular shell it has multiple streams
  - 1 - Success Stream
  - 2 - Error Stream
  - 3 - Warning Stream
  - 4 - Verbose Stream
  - 5 - Debug Stream
  - 6 - Information Stream
  - * - All Streams
- The output can also overwrite file (>) append to file (>>) or redirect the stream to Success for example (>&1)

```
PS C:\> .\script.ps1 > success.log
PS C:\> .\script.ps1 3> error_stream.log
PS C:\> .\script.ps1 *> all_streams.log
```

- Powershell DOES NOT have less than operator for input redirector

Exercise 5

- Try to get all contents of a folder that does not exist (Get-Content command)
  - Get-Childitem c:\notexisistingfolder
  - Get-Childitem c:\users
- Redirect regular output to file output.log and errors to file error.log

Vertical bar (|)

- Even thou on some keyboards the symbol is with the gap in the middle the original character is ASCII 124 and not 0166
- The name everybody knows this symbol is “pipe” since this character is used to ”pipe” the output of one command / script as an entry for the next one
- Important to notice is that since powershell uses objects and not text as command results simple text manipulation is not possible

```
PS C:\> get-service | Where-Object {$_.Name -like "W32*"}

Status   Name               DisplayName
------   ----               -----------
Stopped  W32Time            Windows Time
```

Piping

- How / which objects are passed as input to cmdlets and functions
  - Match output to input
  - -InputObject
  - Parallel vs sequential parsing
- Tee-Object debugging
  - Get-ChildItem -Directory c:\|tee-object -variable debugging|sort FullName

Double / single quote

- Quotation marks are used to specify a literal string. You can enclose a string in single quotation marks (') or double quotation marks (")
- Quotation marks are also used to create a here-string. A here-string is a single-quoted or double-quoted string in which quotation marks are interpreted literally. A here-string can span multiple lines. All the lines in a here-string are interpreted as strings, even though they are not enclosed in quotation marks.
- In commands to remote computers, quotation marks define the parts of the command that are run on the remote computer. In a remote session, quotation marks also determine whether the variables in a command are interpreted first on the local computer or on the remote computer.
- To make double-quotation marks appear in a string, enclose the entire string in single quotation marks
  ‘test “double”’
- You can also enclose a single-quoted string in a double-quoted string
  ”test ‘single’”
- You can aslo double the quotation marks around a double-quoted phrase
  “test “”Double”””

Double / single quote

- To force Windows PowerShell to interpret a double quotation mark literally, use a backtick character. This prevents Windows PowerShell from interpreting the quotation mark as a string delimiter
- Because the contents of single-quoted strings are interpreted literally, you cannot use the backtick character to force a literal character interpretation in a single-quoted string

```
PS C:\> "use quotation mark (`') to begin a string"
use quotation mark (') to begin a string
PS C:\> 'use quotation mark (`") to begin a string'
use quotation mark (`") to begin a string
PS C:\> 'use quotation mark (`') to begin a string'
At line:1 char:24
+ 'use quotation mark (`') to begin a string'
+                        ~
Unexpected token ')' in expression or statement.
At line:1 char:43
+ 'use quotation mark (`') to begin a string'
+                                           ~
The string is missing the terminator: '.
    + CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
    + FullyQualifiedErrorId : UnexpectedToken
```

Double / single quote

- The quotation rules for here-strings are slightly different - A here-string is a single-quoted or double-quoted string in which quotation marks are interpreted literally. A here-string can span multiple lines. All the lines in a here-string are interpreted as strings even though they are not enclosed in quotation marks
- Like regular strings, variables are replaced by their values in double-quoted here-strings. In single-quoted here-strings, variables are not replaced by their values
- You can use here-strings for any text, but they are particularly useful for the following kinds of text
  - Text that contains literal quotation marks
  - Multiple lines of text, such as the text in an HTML or XML
  - The Help text for a script or function document

```
@”
For help, type "get-help”
"@
```

- In single-quoted here-strings, variables are interpreted literally and reproduced exactly

```
@’
The $profile variable contains the path of your Windows PowerShell profile.
'@
```

- In double-quoted here-strings, variables are replaced by their values

```
@”
Even if you have not created a profile,
the path of the profile file is:
$profile.
"@
```

Grave accent / backtick

- The backtick is also called the back apostrophe. If the string is quoted with single quote characters, then the backtick is taken literally and cannot be used to escape any characters
- In Powershell backtick can be used to continue a command on the next line and execute multi line command as one

```
PS C:\> Get-Service * |Sort-Object ServiceType `
>> |Format-Table name, status, `
>> ServiceType, CanStop -auto_
```

- backtick can be used in conjunction with 'n' or 't' to format text with a new line (`n), a tab (`t), carriage return (`r), alert (`a), backspace (`b) or Null (Zero) (`0)

```
PS C:\> Write-Host "Heading `nSub Heading`n`tSecond Sub"
Heading
Sub Heading
        Second Sub
```

- Finally backtick can also be used to escape spaces

```
Get-ChildItem C:\Program` Files\
```

At sign - @

- Officially, @ is the "array operator" but since powershell treats any comma-separated list as an array it is not always necessary
- IT IS necessary with associative arrays where key-value pairs are used
  `@{"Key"="Value";"Key2"="Value2"}`
- You can also wrap the output of a cmdlet (or pipeline) in @() to ensure that the output is a list
- `sp`
-

```
PS C:\> $variable=@(dir c:\windows)
PS C:\> $variable|get-member


   TypeName: System.IO.DirectoryInfo

Name                      MemberType     Definition
----                      ----------     ----------
Mode                      CodeProperty   System.String Mode{get=Mode;}
Create                    Method         void Create(), void Create(System.Security.AccessControl.DirectorySecurity ...
CreateObjRef              Method         System.Runtime.Remoting.ObjRef CreateObjRef(type requestedType)
CreateSubdirectory        Method         System.IO.DirectoryInfo CreateSubdirectory(string path), System.IO.Director...
Delete                    Method         void Delete(), void Delete(bool recursive)
```

- Splatting - Let's you invoke a cmdlet with parameters from an array or a hash-table rather than the more customary individually enumerated parameters

```
PS C:\> $Colors = @{ForegroundColor = "Magenta"; BackgroundColor = "Green"}
PS C:\> Write-Host "This is a test." @Colors
This is a test.
```

Hash

- Hash sign is used for single line comments
  ```
  # this is a comment
  ```
- In case multi line comment is necessary hash symbol is combined with less than / greater than symbols
  ```
  <# this is
  a multiline
  comment #>
  ```
- Can be used to recall history
  - # (tab) to recall the last command
  - #string (tab) to recall commands from history containing the string

Dollar sign

- Dollar sign is used to declare or use a variable

```
$variable=111   #declares a variable
$variable       #uses/displays the contents of variable
```

Slash /

- Powershell uses .NET framework and based on **Path.AltDirectorySeparatorChar Field** it allows both- slash (UNIX style) and backslash (Windows style) directory separators but problems might occur since forward slash might need additional escaping
- Slash character also has the division operator meaning
- Slash may also be a date separator depending on the operating system Date / Time formatting

Backslash \

- Powershell uses .NET framework and based on **Path.AltDirectorySeparatorChar Field** it allows both- slash (UNIX style) and backslash (Windows style) directory.
- Backslash is also used in regular expressions in powershell -
  - \w matches any word character, meaning letters and numbers
  - \s matches any white space character
  - \d matches any digit character

```
PS C:\Users\kasutaja> "randomtext" -match "\d"
False
PS C:\Users\kasutaja> "randomtext" -match "\w"
True
```

  - Additional usage in regular expressions topic on next slide

Regular expressions in Powershell

- ^ - By default, the match must occur at the beginning of the string; in multiline mode, it must occur at the beginning of the line
- $ - By default, the match must occur at the end of the string or before \n at the end of the string; in multiline mode, it must occur at the end of the line or before \n at the end of the line
- \A - The match must occur at the beginning of the string only (no multiline support)
- \Z - The match must occur at the end of the string, or before \n at the end of the string
- \z - The match must occur at the end of the string only
- \G - The match must start at the position where the previous match ended
- \b - The match must occur on a word boundary
- \B - The match must not occur on a word boundary
- \w+ - Match one or more word characters
- \w* - Match zero or more word characters
- \s? - Match zero or one space
- \n – Match a newline character

EXAMPLES

- ((\w+(\s?)){2,} - Match one or more word characters followed either by zero or by one space exactly two times. This is the first capturing group. This expression also defines a second and third capturing group: The second consists of the captured word, and the third consists of the captured spaces
- ,\s - Match a comma followed by a white-space character
- (\w+\s\w+) - Match one or more word characters followed by a space, followed by one or more word characters. This is the fourth capturing group
- \s\d{4} - Match a space followed by four decimal digits
- (-(\d{4}|present))? - Match zero or one occurrence of a hyphen followed by four decimal digits or the string "present". This is the sixth capturing group. It also includes a seventh capturing group

Underscore character _

- Typically underscore character is interpreted as regular character
- Special variables with underscore – for example $_ has the meaning of “in this pipeline” so you can use it to avoid repeating the same command – for example:

```
PS C:\Users\kasutaja> get-service | Where-Object {$_.name -match "win"}

Status   Name               DisplayName
------   ----               -----------
Running  WinDefend          Windows Defender Service
Running  WinHttpAutoProx... WinHTTP Web Proxy Auto-Discovery Se...
Running  Winmgmt            Windows Management Instrumentation
Stopped  WinRM              Windows Remote Management (WS-Manag...
```

- In this case the $_ is substitution for Get-Service
- More about special character combinations are described in another slide

Asterisk character

- Asterisk character is one of the wildcard operators
  - * - Matches zero or more characters
    - a* matches “aA” and “apple” but not “banana”

Question mark

- Question mark is one of the wildcard operators that matches exactly one character in that position
  - ?n matches “in”, “on” but does not match “ran”

Exclamation mark

- Exclamation mark in powershell has the –not operator or “not equal” operator meaning
- For example:
  ```
  ![bool]0 # True
  ![bool]1 # False
  ![bool]"" # True
  ![bool]"test" # False
  ![bool]$null # True
  ![bool](1>2) # True
  ```

Percent sign

- In powershell the % sign is an alias for the command “foreach” or more specifically ForEach-Object
- In mathematical approach the % is the modulus operator
  - For example 12%5 result is 2
- The percent sign can also be used to assign the modulo to an variable
  - $variable= 13 #assign the value of 13
    $variable %=5 #get the modulo after dividing by 5
    $variable # print out the modulo
    3

Ampersand

- Ampersand in powershell is a call operator which allows the execution of a script, command or a function
- For example & “[PATH] command” [arguments]
- & "C:\folder\yourprogram.exe" argument1 argument2
- Call operator runs commands in separate scope instead of Dot sourcing where commands are ran in same scope

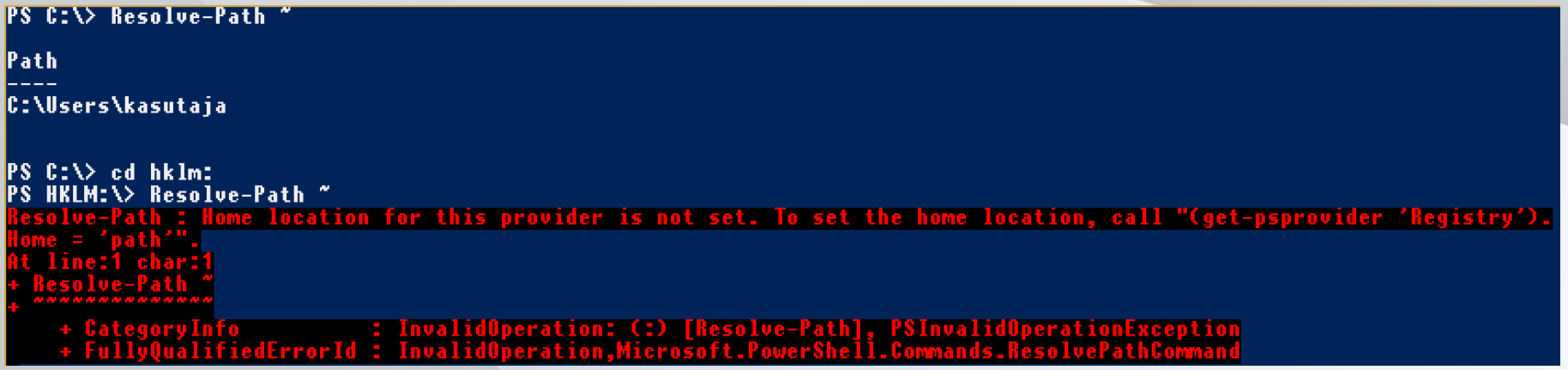
```
PS C:\> $x=1
PS C:\> &{$x=2};$x
1
PS C:\> . {$x=2};$x
2
```

Caret

- Caret can be used to make regular expression matches
- Match anything but these- will match any characters except those with caret in square brackets
  - “testing stuff” -match “testin[^abc] stuff” returns True
- Match anything but these in a range with caret
  - “abc” -match “[^abc-ijk-xyz]” returns False
  - “abc” -match “[^i-z]” returns True
- Match from the beginning of the line
  - “no money no honey” -replace “^no”,”” returns ”money no honey”
- Important to notice is that caret is used as an escape character in windows CMD shell !

Tilde

- Tilde ~ in powershell stands for the profile directory of the current user
- Depending on the DRIVE the ~ can point to diferent locations- on file system drive it has the same value as in linux - home folder (in powershell $home) but in registry or cert drive it might refer to diferent location



Colon

- Colon has a special meaning in PowerShell variable names: It is used to associate the variable with a specific *scope* or namespace. There are a few scopes defined by PowerShell, such as script and global
  - for example a global variable might be named $global:var
- Colon can also be used to specify items on a PSDrive $Env:Foo
  - For example you can use ${c:\textfile.txt} = sometext

```
PS C:\> ${c:\textfile.txt} = "Sometext"
PS C:\> Get-Content C:\textfile.txt
Sometext
```

- Double colon can also be used to call static methods of a .NET class

```
PS C:\> $test=[string]
PS C:\> $test::join('+',(1,2,3))
1+2+3
```

```
PS C:\> $number=1.234567890
PS C:\> [System.Math]::Round($number,3)
1,235
```

- Or static property

```
PS C:\> [int]::MaxValue
2147483647
```

Semicolon

- In powershell semicolon ; is a command separator or statement terminator (as & is in CMD shell)

```
PS C:\> get-date -DisplayHint date ;get-date -DisplayHint time

10. juuli 2018
15:42:20
```

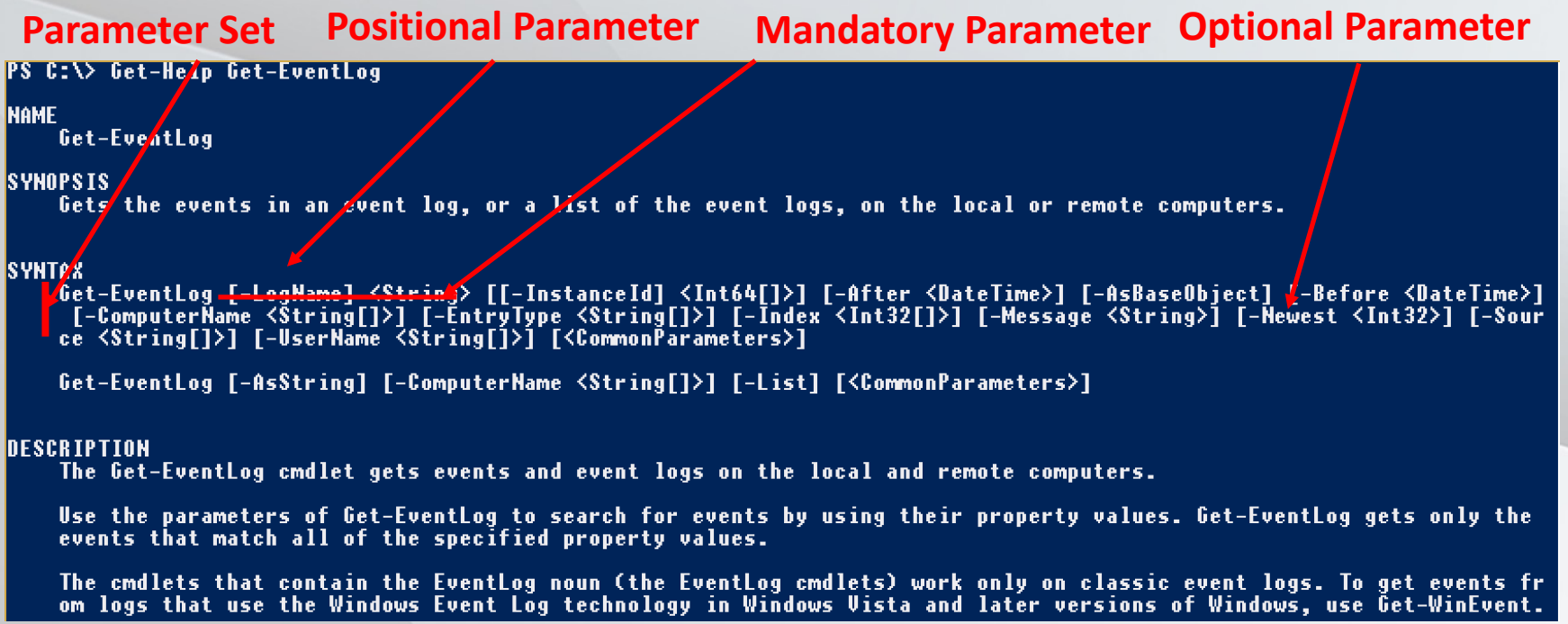
Special tokens in powershell

- $$ (double dollar) – last token of last command
  - Write-Host “test”
    $$
    test
    test
- $^ (dollar caret) – first token of last command
  - Write-Host “test2”
    $^
    test2
    Write-Host
- $? (dollar question mark) – Returns True or False indicating if the previous command ended with an Error
- $_ (dollar underscore) – Refers to the item inside foreach loop, newer Poershell implemented $PSItem instead
- .. (double dot) – specify a range
- … (triple dot) – Operator (..) combined with fractional value prefix (.)
  - 1…3 all integer values between 1 and 0.3
- :: (double colon) – Reference to static member of a class
- += (plus equals) – Increments value on the left by the amount on the right and stores the result. For strings it concatenates the strings. Add values to arrays
- --% (double minus percentage) – stops powershell style parsing and allows to feed in commands with special characters without parsing them
  - Write-Host --% %PATH%,and=$some{additions}
    --% %PATH%,and=$some{additions}
    Write-Host %PATH %,and=$some{additions}
    %PATH% and= additions
- Int with KB,MB,GB,TB,PB – kilobytes,megabytes,gigabytes,terabytes,petabytes

Get-Help

- For getting help about command
- To get list of all cmdlets, providers and aliases Get-Help *
- To get help about commands matching specific pattern
  Get-Help Get-*
  Get-Help *-service
  Get-Help *-AD*
- To update the help files from Microsoft type Update-Help
- Get-Help also has aliases help and man
- Get-Help about*
- Get-Help Get-Eventlog -Full

Interpreting Get-Help

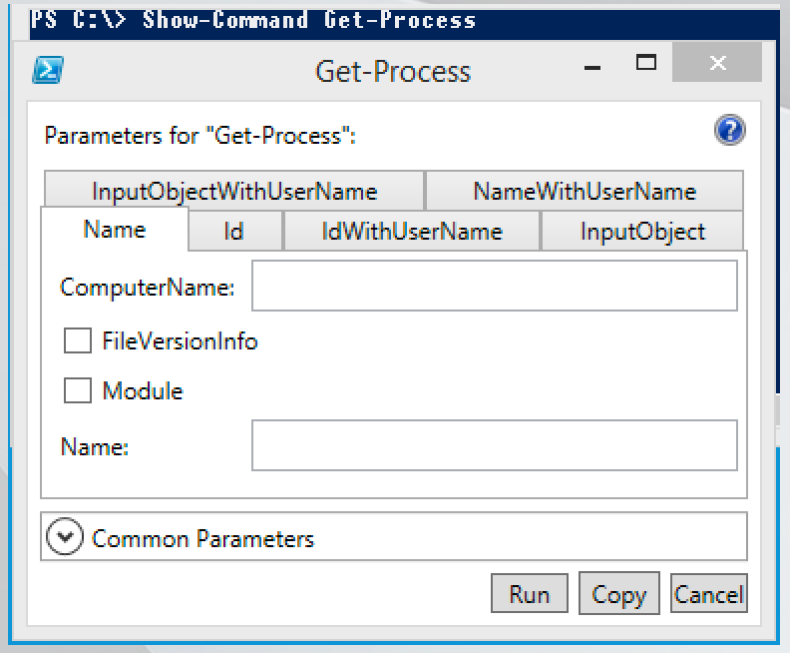


Powershell aliases

- Get-Alias
- Get-Alias ls
- New-Alias blah calc.exe
- New aliases are not permanent (profile)

Show-Command

- Brings up a GUI window where user can enter the commands parameters
- Each parameter set is shown on a separate tab. This makes it visually clear that parameters cannot be mixed and matched between sets



Classes Objects Properties and Methods

- Class = Object type (constructor)
- Object has Properties and Methods
  - Properties represent the attributes
  - Methods represent the actions

```
PS C:\> $test=Get-Process -Name calc
PS C:\> $test

Handles  NPM(K)    PM(K)      WS(K) VM(M)   CPU(s)     Id ProcessName
-------  ------    -----      ----- -----   ------     -- -----------
    115      28     7292      13272   108     0,34   3852 calc


PS C:\> $test.Name;$test.ID;$test.Company
calc
3852
Microsoft Corporation
```

Get-Member (alias gm)

- Get-Process |Get-Member - displays all the properties and methods what the first item in the array has

```
PS C:\> $test|Get-Member


   TypeName: System.Diagnostics.Process

Name                       MemberType     Definition
----                       ----------     ----------
Handles                    AliasProperty  Handles = Handlecount
Name                       AliasProperty  Name = ProcessName
NPM                        AliasProperty  NPM = NonpagedSystemMemorySize
PM                         AliasProperty  PM = PagedMemorySize
VM                         AliasProperty  VM = VirtualMemorySize
WS                         AliasProperty  WS = WorkingSet
Disposed                   Event          System.EventHandler Disposed(System.Object, System.EventArgs)
ErrorDataReceived          Event          System.Diagnostics.DataReceivedEventHandler ErrorDataReceived(System.Objec...
Exited                     Event          System.EventHandler Exited(System.Object, System.EventArgs)
OutputDataReceived         Event          System.Diagnostics.DataReceivedEventHandler OutputDataReceived(System.Obje...
BeginErrorReadLine         Method         void BeginErrorReadLine()
BeginOutputReadLine        Method         void BeginOutputReadLine()
CancelErrorRead            Method         void CancelErrorRead()
CancelOutputRead           Method         void CancelOutputRead()
Close                      Method         void Close()
CloseMainWindow            Method         bool CloseMainWindow()
CreateObjRef               Method         System.Runtime.Remoting.ObjRef CreateObjRef(type requestedType)
Dispose                    Method         void Dispose(), void IDisposable.Dispose()
Equals                     Method         bool Equals(System.Object obj)
GetHashCode                Method         int GetHashCode()
GetLifetimeService         Method         System.Object GetLifetimeService()
GetType                    Method         type GetType()
InitializeLifetimeService  Method         System.Object InitializeLifetimeService()
Kill                       Method         void Kill()
Refresh                    Method         void Refresh()
Start                      Method         bool Start()
ToString                   Method         string ToString()
WaitForExit                Method         bool WaitForExit(int milliseconds), void WaitForExit()
WaitForInputIdle           Method         bool WaitForInputIdle(int milliseconds), bool WaitForInputIdle()
__NounName                 NoteProperty   System.String __NounName=Process
BasePriority               Property       int BasePriority {get;}
Container                  Property       System.ComponentModel.IContainer Container {get;}
EnableRaisingEvents        Property       bool EnableRaisingEvents {get;set;}
ExitCode                   Property       int ExitCode {get;}
ExitTime                   Property       datetime ExitTime {get;}
Handle                     Property       System.IntPtr Handle {get;}
HandleCount                Property       int HandleCount {get;}
HasExited                  Property       bool HasExited {get;}
Id                         Property       int Id {get;}
```

Commands that affect and modify the System

- Many commands that change the system state like stopping or starting something, restarting, disabling/ enabling users etc have two special parameters:
  - -WhatIf # this command will not actually execute- instead it will display messages what would have done
  - -Confirm # this command will stop and ask to continue for each item it is about to modify- for example for disabling 10 users it will prompt to confirm 10 times
- Both of the commands have an internal setting called *confirm impact* which is set by the command developer to Low,Medium or High
- By default Powershell has preference variables for both $ConfirmPreference and $WhatIfPreference set to High
  - If the command confirm impact is higher or equal than the corresponding preference variable execution will halt and prompt for confirmation
  - If the command confirm impact is lower than the preference the command is automatically executed

Drives

- Get-PSDrive displays different drive containers available
- New-PSDrive –name test –psprovider filesystem –root \\localhost\c$
- Get contents of different drives
  Dir variable:\
  Dir env:\
- Manipulate items in drive container
  New-Item
  Remove-Item
  Copy-Item
  Rename-Item
- Access the properties of the items
  Get-Item
  Get-Childitem
  Set-Item
  Clear-Item
- Access the individual property of an item
  Get-Itemproperty
  Set-Itemproperty
  Clear-Itemproperty

```
PS C:\> Get-ItemProperty $home |Format-List *


PSPath            : Microsoft.PowerShell.Core\FileSystem::C:\Users\kasutaja
PSParentPath      : Microsoft.PowerShell.Core\FileSystem::C:\Users
PSChildName       : kasutaja
PSDrive           : C
PSProvider        : Microsoft.PowerShell.Core\FileSystem
BaseName          : kasutaja
Mode              : d----
Name              : kasutaja
Parent            : Users
Exists            : True
Root              : C:\
FullName          : C:\Users\kasutaja
Extension         :
CreationTime      : 27.01.2014 14:18:44
CreationTimeUtc   : 27.01.2014 12:18:44
LastAccessTime    : 10.07.2018 12:06:44
LastAccessTimeUtc : 10.07.2018 9:06:44
LastWriteTime     : 10.07.2018 12:06:44
LastWriteTimeUtc  : 10.07.2018 9:06:44
Attributes        : Directory
```

Environment Variables

- Dir env:\
- New-item env:\newitem -value “First value”
- Set-Item env:\newitem -value “Second value”
- Remove-Item env:\newitem
- Display certain variable $env:PATH $env:COMPUTERNAME

Customizing powershell

- Profile scripts reside in $Profile
  $profile | Format-List * -Force

```
PS C:\> $profile |format-list * -force


AllUsersAllHosts       : C:\Windows\System32\WindowsPowerShell\v1.0\profile.ps1
AllUsersCurrentHost    : C:\Windows\System32\WindowsPowerShell\v1.0\Microsoft.PowerShell_profile.ps1
CurrentUserAllHosts    : C:\Users\kasutaja\Documents\WindowsPowerShell\profile.ps1
CurrentUserCurrentHost : C:\Users\kasutaja\Documents\WindowsPowerShell\Microsoft.PowerShell_profile.ps1
```

- The scripts are executed in the same order so starting with AllUsersAllHosts and finally CurrentUserCurrentHost
- $host and $host.UI.RawUI

Functions

- Function is a block with a name
- To view available functions list the contents of the function drive
  dir function:\
- You can also view the contents of a function
  Get-content function:\more
- To load additional functions dot sourcing can be used
  . C:\script\folder\script.ps1
  . .\scriptincurrentfolder.ps1
  . \\script\in\network\folder\script.ps1
- Naming the functions and overlapping

Modules

- What are modules and cmdlets
- Where does powershell store the modules
- Get-Module -ListAvailable
- Import-Module -Name ActiveDirectory
- Get-Command -Module ActiveDirectory
- Get-Module -All
- Import-Module –Name x:\path\to\module.psm1

Powershell Gallery

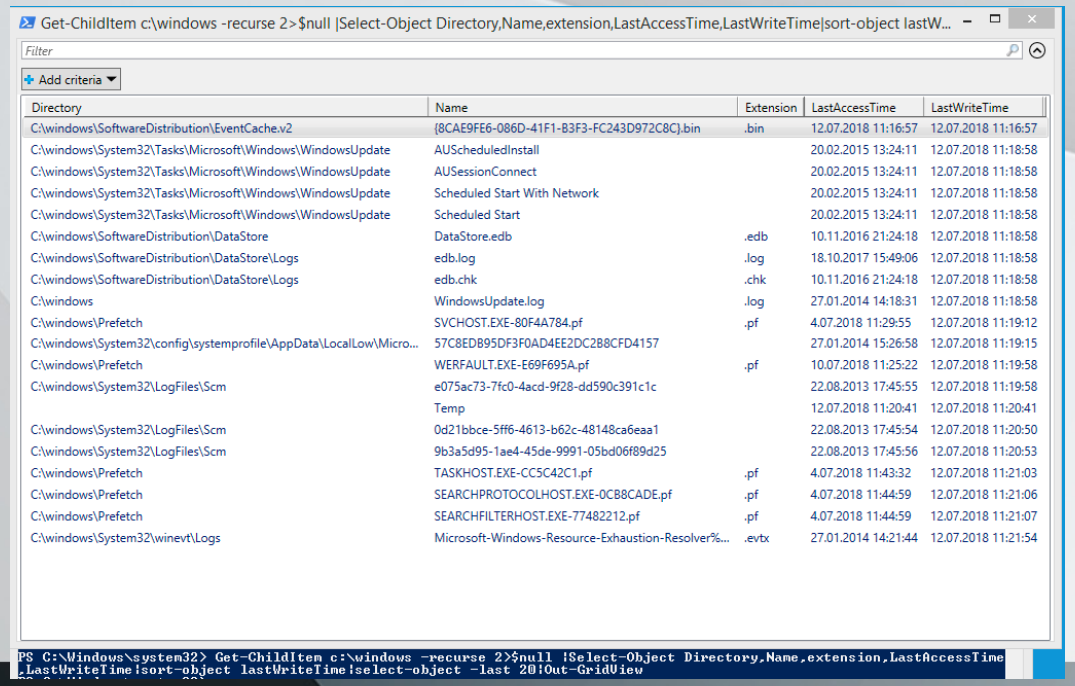
- Biggest public repository for powershell modules www.powershellgallery.com
- WMF 5 includes commands to search and download modules from the gallery
  - Find-Module
  - Find-Script
  - Save vs Install module
- NuGet
  - Get-Command -Module PowershellGet
  - Get-Command -Module PackageManagement
  - Get-PackageProvider |Format-Table Name,ProviderPath

Select-Object

- Modifier when piping objects – helps to select only necessary properties
- Get-service win* | select-object Name,Status |out-gridview
- Select-String - powershell “grep / findstr”

Sort-Object

- Sort-Object default alias Sort
  - Allows sorting of objects based on one or multiple properties
  - Get-ChildItem c:\windows -recurse 2>$null |Select-Object Directory,Name,extension,LastAccessTime,LastWriteTime|sort-object lastWriteTime|select-object -last 20|Out-GridView



Where-Object

- $PSItem or $_ is a variable for each object that is received from the pipe and curly brackets define the condition that must return True or False
- Get-Service | Where-Object {$PSItem.status -eq “running”}

```
PS C:\> Get-Service | Where-Object {$PSItem.status -eq "running"}

Status   Name               DisplayName
------   ----               -----------
Running  Appinfo            Application Information
Running  Apple Mobile De... Apple Mobile Device
Running  AppXSvc            AppX Deployment Service (AppXSVC)
Running  AudioEndpointBu... Windows Audio Endpoint Builder
```

- Get-Process | w
- Get-Command | where {$PSItem.name -Notmatch “^[SG]et.”}

Arrays in powershell

- By default arrays are definded with @ but powershell also guesses if you want to create an array
  - $empty=@()
  - $one=@(“1item”)
  - $many=(“one”,2,”three”)
  - $many +=“add_one_item”
  - $sequence= (1..100)
  - $valuepair=@{value1=“one”;value2=“two”}
- Working with array
  - $sequence[0] – first
  - $sequence[-1] – last
  - $new1,$new2,$new3=$many[0,1,2]

Directing command output to array

- $history=get-history
- $connections=netstat.exe -ano
- $external=python.exe pythonscript.py
- $external_to_array=@(python.exe \path\to\pythonscript.py)
- $content=@(Get-Content c:\some\textfile.txt)
- $binary=Get-Content executable.exe –encoding byte

Loops and cycles: Do While / Until

- Simple looping:
  Other commands
  Do something
  　　Change something else
  While something meets the criteria
- $i=1
  DO {
  　Write-Host ”Current something $i”
  　$i++
  } While ($i -le 10)
- Until is the opposite of While (while runs while $true, until runs until $true)

While conditions

- Arithmetic conditions:
  - -gt
  - -lt
  - -eq
  - -le
  - -ge
  - -approx
- String conditions:
  - -contains
  - -like
  - -notlike
  - -in
  - -notin

- Logical / bitwise:
  - -(b)and
  - -(b)or
  - -(b)xor
  - -(b)not
- Object conditions
  - -match
  - -nomatch
- Add c in front of contition to make it case sensitive
  - -ceq
  - -clike

Loops and cycles: While

- While loop has the condition in the beginning
  - Killing processes immediately
- While (condition) { do something}
- While ($true) {
  Stop-Process -Name notepad -ErrorAction SilentlyContinue
  Sleep 5
  }

Loops and cycles: For

- Simple for cycle:
  Other commands
  For as long as (initial value; end condition; change in the end of iteration)
    Change or Do something
  Continue with normal flow
- For ($i=1;$i -le 10;$i++) {
  Write-Host ”Current value $i”
  }
- Continuous loop:
  For () { Write-Progress $(Get-Date) }

Loops and cycles: Foreach

- Foreach (pipeline version Foreach-Object together with $_)
- Foreach ($item in $allitems) {Do something with $item}
- $processes=Get-Process
  Foreach ($process in $processes)
  {
  Write-Host “ID $process.id is $process.name”
  }
- Get-Process | ForEach-Object {Write-Host $_.Name}

Flow control: If..Else..ElseIf

- Typical construct:
  Other commands
  If (condition) {Do something}
  ElseIf (another condition) {Do something else}
  Else {Do something if previous statements didn't match}
- For example:
  $i=5
  If ($i –gt 10) {Write-Host "Number bigger than 10"}
  ElseIf ($i -lt 5) {Write-Host "Number smaller than 5"}
  Else {Write-Host "Number is $i"}

Flow control: Switch

- In Switch the value to compare against is placed inside the parenthesises and then compared against the statements in curly brackets
- Example:
  Other commands
  Switch (value to compare) {
    {first condition} {Do something}
    {second condition} {Do something else}
    {Nth condition} {Do something else than else}
  }
- Switch can have additional options to specify the match criteria
  - -Case Forces case sensitive match- by default the matching is case insensitive
  - -Wildcard Converts the value to compare against into a string and allows wildcard usage like * or ?
  - -RegEx compares the string against regular expression

Flow control: Switch

- $dll=$exe=$txt=0
  switch(Get-ChildItem c:\windows) {
    {$_.Extension -like ".dll"} {$dll++}
    {$_.Extension -like ".exe"} {$exe++}
    {$_.Extension -like ".txt"} {$txt++}
  }
  Write-Host "DLL files $dll `nEXE files $exe `nTXT files $txt"

Export and import objects

- Export-CSV
- Import-CSV
- Export-CLIXML
- Import-CLIXML
- ConvertTo-JSON
- ConvertFrom-JSON
- ConvertTo-HTML

Formatting outputs

- Default formatting
- Wide lists (Format-Wide)
- Lists (Format-List)
- Tables (Format-Table)
  - GroupBy
- Select-Object / -Property

Outputters

- Out-Default
- Out-Host
- Out-File
- Out-Gridview
- Out-String
- Out-Null
- Out-Printer

Exercise 6

- Get all services that are running on **DC1**
- Get the executable path and store them in a variable
  - You might need to access
  - Registry
  - Wmi
  - Command shell
- Colour the service name red and executable with path green

Exercise 7

- In **WS1** execute notepad and if installed also notepad++
- From all processes in workstation WS1 find the process names that contain ”pad” in the name and show the path of the executable

Bonus Exercise 8

- Find all CA-s in current computer **(DC1)** that are not trusted by Microsoft
  - Get Microsoft officially trusted CA-s
  - Get all locally trusted CA-s
  - Display the certificate(s) which are not trusted by Microsoft